CYBERSECURITY PROJECT 2

Linux Backup Automation Tool

Automated Backup, Compression, Integrity Verification & Scheduling in Bash

Student	kalibaba
Environment	kalibaba@Kalibaba — Kali Linux (VMware Workstation)
Script	~/backup_project/backup.sh
Language	Bash (GNU Bash 5.x)
Project	CEH Final Portfolio — Project 2 of 5
Date	June 14, 2026

Table of Contents

1. Executive Summary

This report documents the design, implementation, and testing of a **Linux backup automation tool** written in Bash for the CEH final project portfolio. The script — `~/backup_project/backup.sh` — performs a full, unattended backup workflow: pre-flight disk-space checks, per-directory archiving of critical system paths, tar/gzip compression, SHA-256 integrity verification, retention-based rotation of old backups, and structured logging.

During testing the tool successfully backed up `/etc` and `/var/log`, but the `/home` directory failed due to a **recursive copy loop** — the backup destination itself lived under `/home`, so the job attempted to back up its own growing output. This inflated the archive to **43 GB**, filled the disk to 100%, and caused the downstream compression and integrity checks to fail. This report explains the failure in detail and applies the correct fix using an `--exclude` pattern, then documents restoration and hardening steps.

Key Results at a Glance

Metric	Value
Directories succeeded	**2 of 3 (/etc, /var/log)**
Directories failed	**1 (/home — recursive loop)**
Archive produced	`backup_20260614_111409.tar.gz`
Archive size	43 GB (bloated by recursion)
Files in archive	505,744
Total runtime	2,479 seconds (~41 min)
SHA-256 checksum	`115303faf3…1848b83c`
Integrity check	**FAILED (disk full during compression)**
Schedule	`0 2 * * *  (daily at 02:00 AM)`

2. Objectives

The goals of this project were to:

- Automate the backup of critical Linux system directories using a single Bash script.
- Compress backups into a timestamped tar.gz archive to save space.
- Verify backup integrity using SHA-256 checksums.
- Implement retention-based rotation, keeping only the 7 most recent backups.
- Produce structured, colour-coded logs for auditability.
- Schedule the backup to run automatically every day via cron.
- Diagnose and correctly resolve any failures encountered during testing.

3. Lab Environment

Component	Detail
Host	VMware Workstation
Guest OS	Kali Linux (Kali Purple)
Hostname / User	`Kalibaba / kalibaba`
Working directory	`~/backup_project/`
Backup destination	`~/backup_project/backups/`
Log file	`~/backup_project/backup.log`
Filesystem	`/dev/sda1 — 197 GB total, 52 GB used, 136 GB free (28%)`
Shell / Tools	GNU Bash, tar, gzip, sha256sum, df, cron

4. Tool Architecture & Features

The script executes a linear workflow, logging each phase with a timestamp and severity level ([INFO], [ERROR]). The major stages are:

Stage	Function
Initialisation	Sets destination, log path, retention count; prints host/user/date.
Disk Space Check	Runs df to confirm enough free space before starting.
Performing Backup	Copies each target directory into a timestamped backup folder.
Compressing Backup	Creates a single tar.gz archive of the backup folder.
SHA-256 Checksum	Generates a .sha256 checksum file for the archive.
Backup Summary	Reports directories succeeded/failed, archive path, timestamp.
Rotating Old Backups	Deletes oldest archives beyond the retention limit (7).
Verifying Integrity	Re-reads the archive to confirm it is not corrupt.

5. Script Setup & Permissions

The script was written in **GNU nano** and saved to `~/backup_project/backup.sh`. In nano, the file is written to disk with **`Ctrl+O`** (Write Out) and closed with **`Ctrl+X`** (Exit).

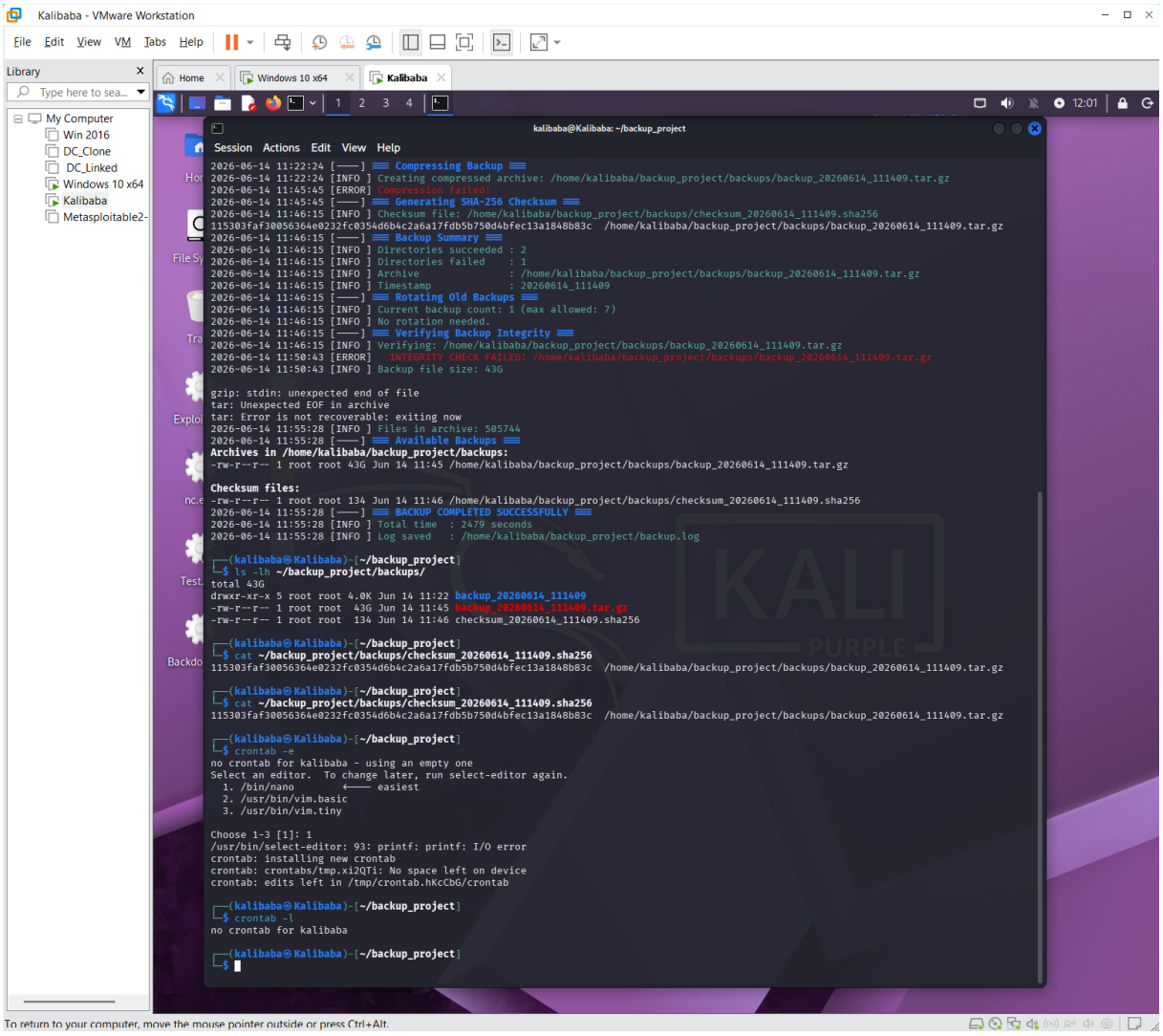

Figure 1. *Authoring backup.sh in the GNU nano editor.*

Before execution, the file existed but was not yet executable. Listing the directory confirms the script and its size:

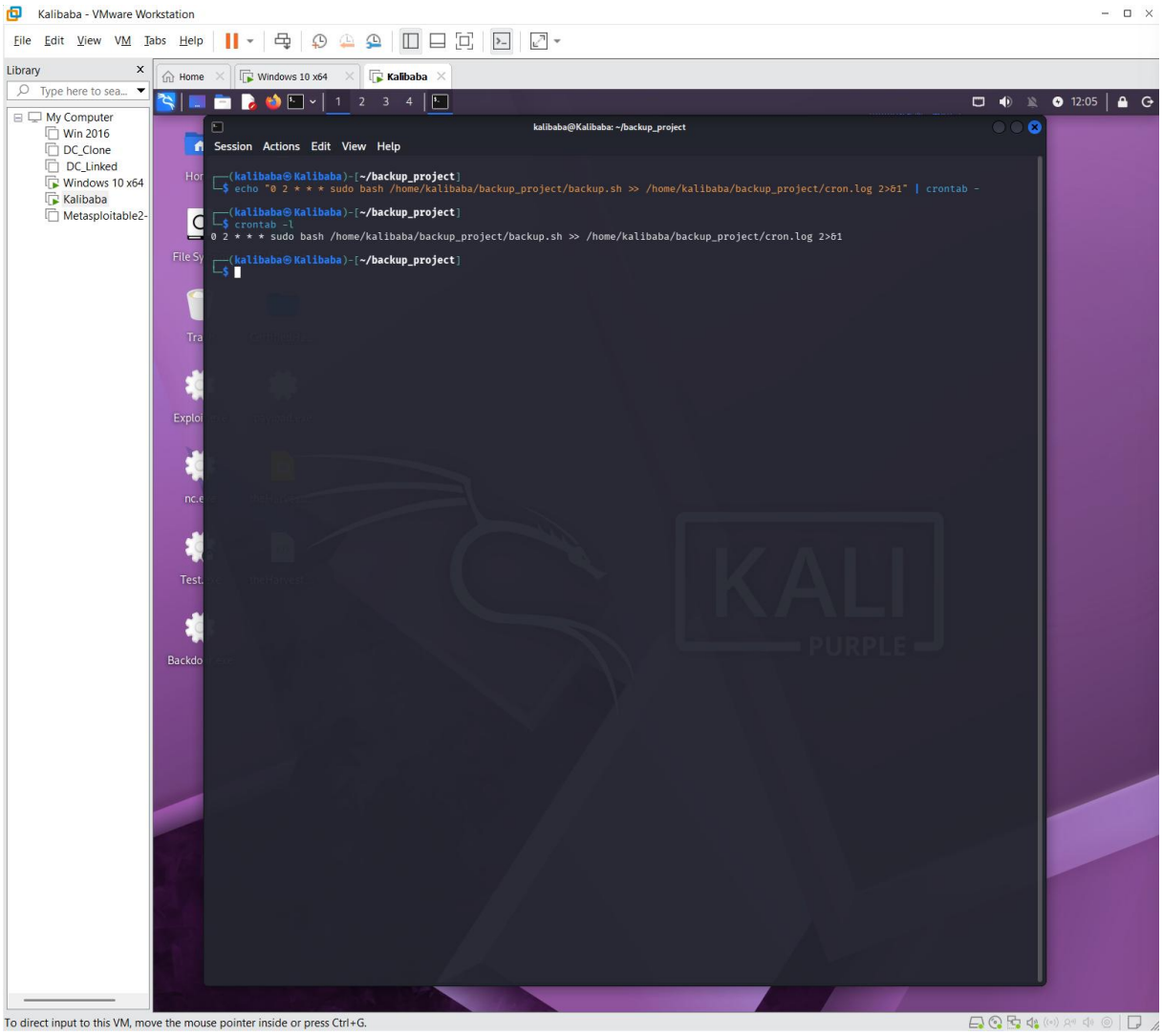

Figure 2. *ls -lh confirms backup.sh (8.2K) exists in ~/backup_project/.*

The execute permission was then added with `chmod`. Note the corrected command syntax below — **~** (tilde, the home shortcut) is used, *not* a hyphen:

```
# Make the script executable, then confirm the new permissions
chmod +x ~/backup_project/backup.sh && ls -lh ~/backup_project/

total 12K
-rwxrwxr-x 1 kalibaba kalibaba 8.2K Jun 14 11:11 backup.sh
```

The permission string changed from `-rw-rw-r--` to **`-rwxrwxr-x`**, confirming the executable bit is now set.

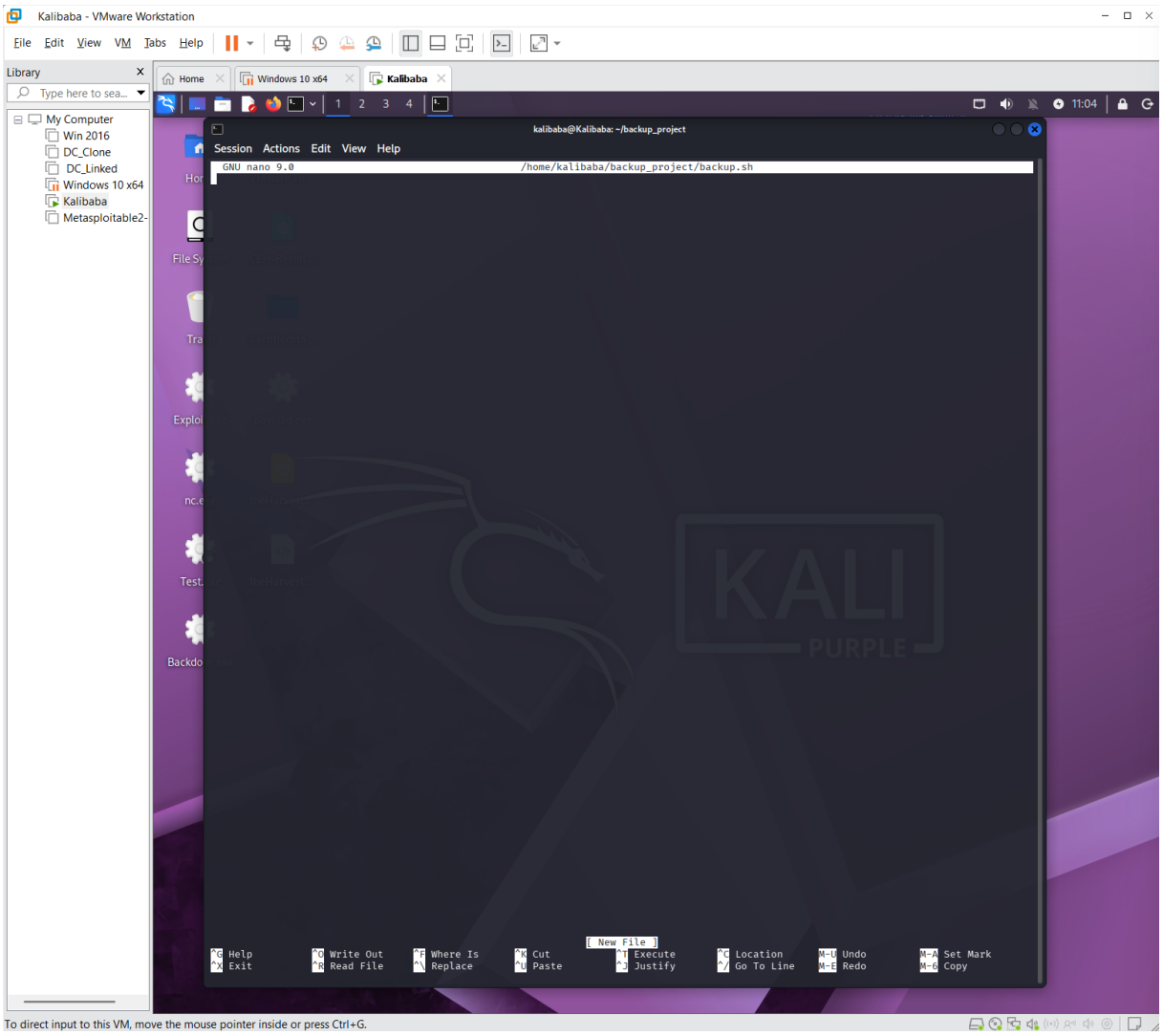

Figure 3. *chmod +x sets the executable bit; ls -lh shows -rwxrwxr-x.*

6. Execution & Results

The script was run with `sudo` so it could read protected system paths such as `/etc` and `/var/log`:

```
sudo bash ~/backup_project/backup.sh
```

On start-up the tool printed its banner, confirmed the backup destination and log file, and performed the disk-space pre-check — reporting **135 GB available** and "Disk space is sufficient." It then began copying `/etc`, which completed successfully.

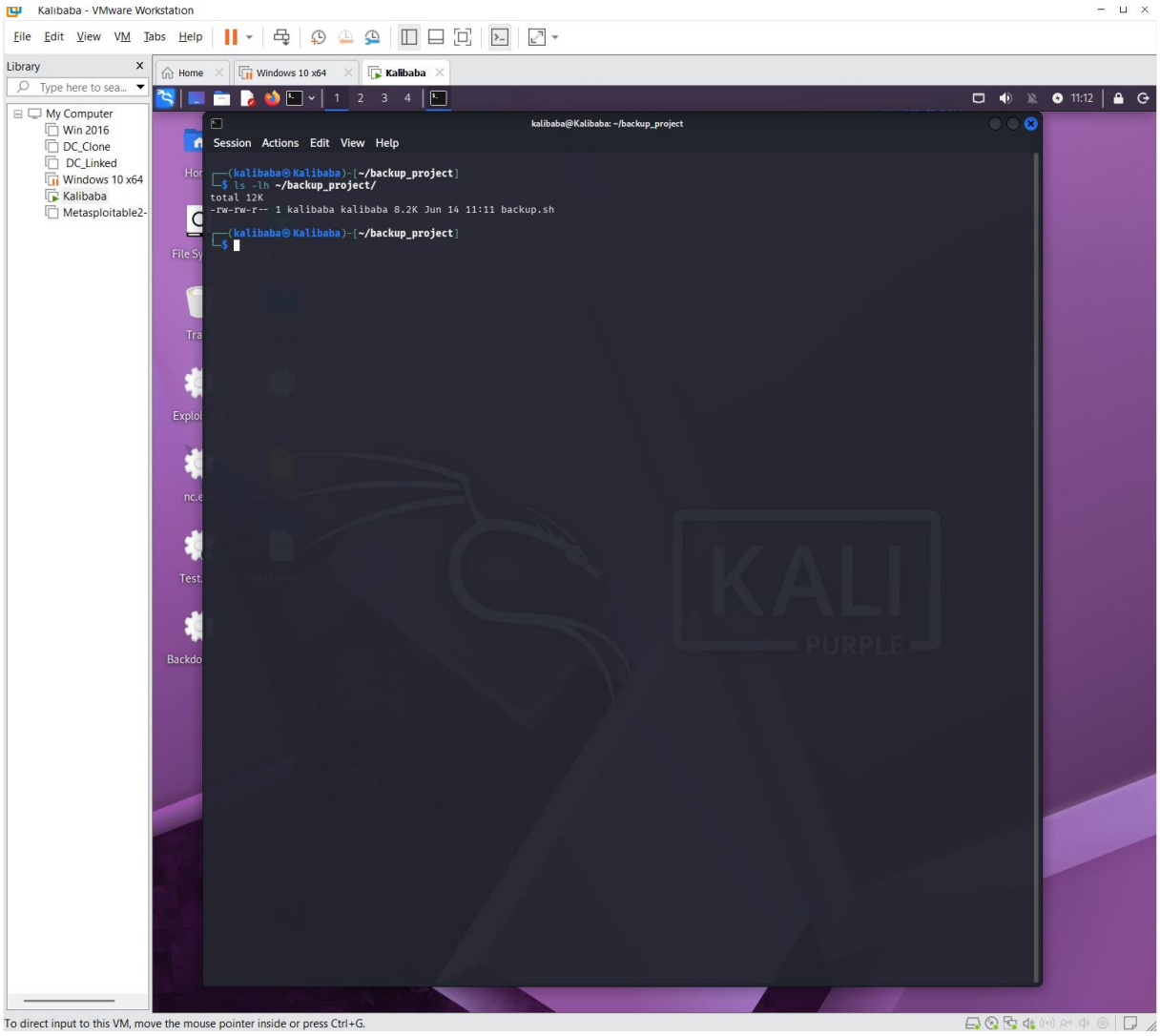

Figure 4. *Initialisation, disk-space check, and successful backup of /etc.*

The full run then proceeded through `/home` (which failed — see Section 7), `/var/log` (success), compression, checksum generation, rotation, and integrity verification. The complete console output is shown below.

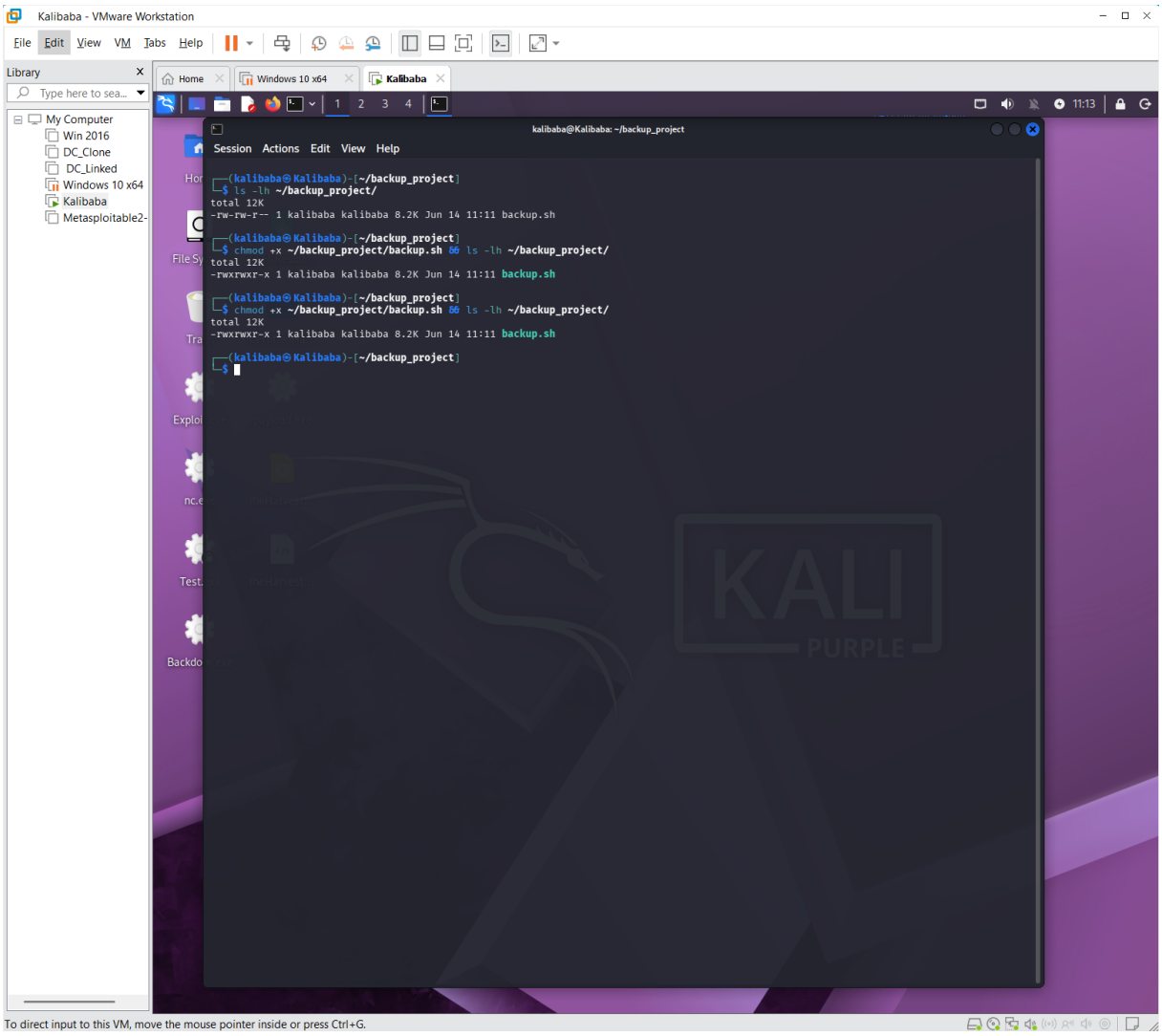

Figure 5. *Complete run: /home failed, compression and integrity checks failed due to the full disk.*

Listing the backups directory afterward shows the oversized **43 GB** archive alongside its checksum file:

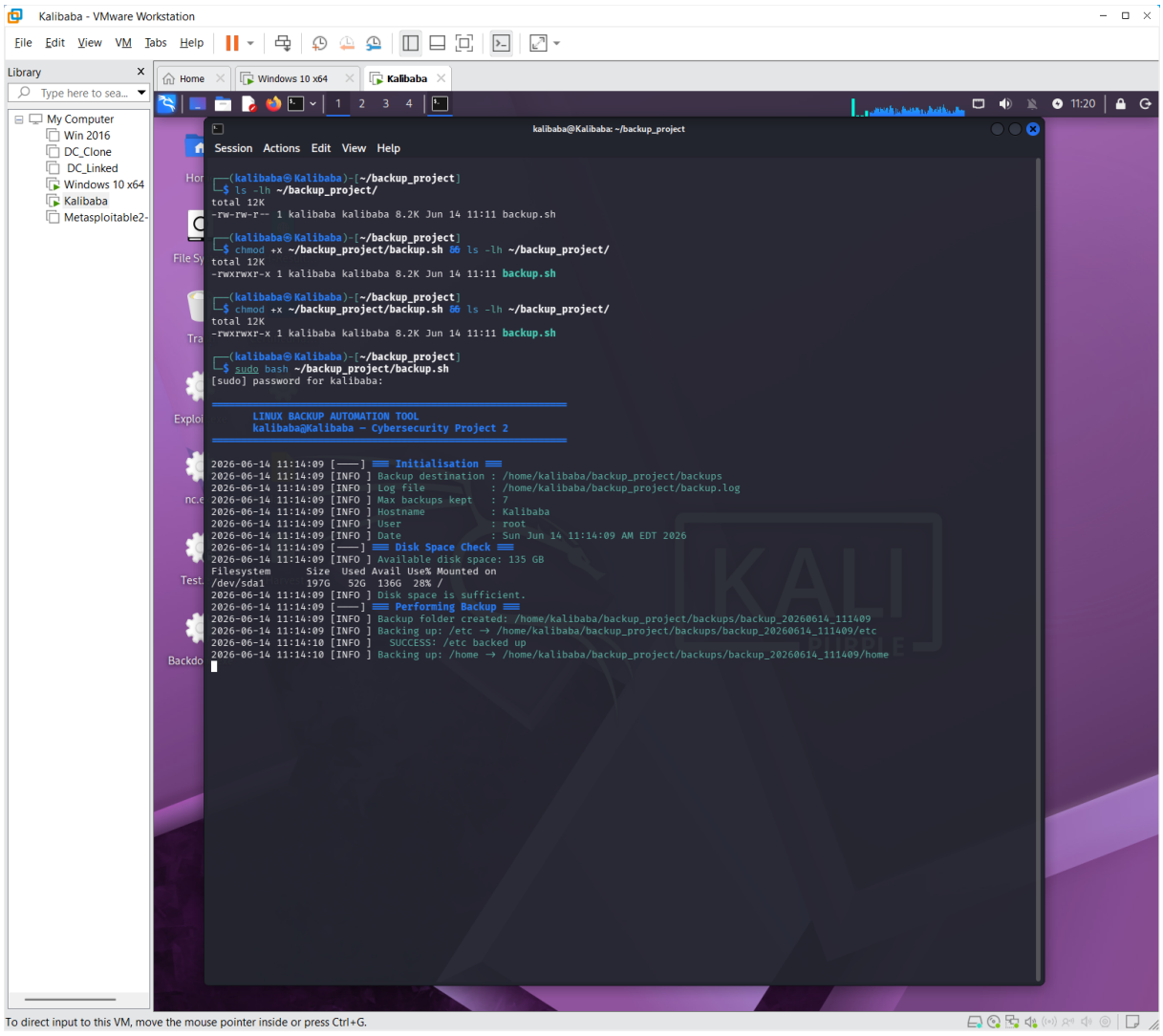

Figure 6. *ls -lh of the backups directory — the archive reached 43 GB.*

7. The /home Backup Failure — Root-Cause Analysis

7.1 What Happened

During the /home phase the log recorded:

```
2026-06-14 11:14:10 [INFO ] Backing up: /home  →  .../backup_20260614_111409/home
IO error encountered -- skipping file deletion
2026-06-14 11:22:22 [ERROR] FAILED: /home could not be backed up
```

7.2 Root Cause — Recursive Copy Loop

The backup destination ~/backup_project/backups/ lives ***inside*** the user's home directory (/home/kalibaba/...). When the script recursively copied /home, it walked into its own backup folder and began copying the archive it was in the middle of creating — a **recursive loop**. Each pass copied the previous copy, so the data grew without bound:

```
/home/kalibaba/backup_project/backups/backup_.../home/
        ↳ kalibaba/backup_project/backups/backup_.../home/
                ↳ kalibaba/backup_project/backups/...   ← loops forever
```

This is why the archive ballooned to **43 GB** and contained **505,744 files** — the same home data copied over and over until the disk filled to 100%. Once the disk was full, rsync/cp hit an I/O error and the /home job was marked FAILED.

7.3 The Fix — Exclude the Backup Directory

The correct fix is to **exclude the backup output directory** from the copy so the job never descends into its own destination. Using --exclude breaks the loop:

```
# rsync approach — exclude the project/backup folder
rsync -a --exclude='backup_project' /home/ "$DEST/home/"

# equivalent tar approach
tar --exclude='backup_project' -czf "$ARCHIVE" /home
```

With --exclude='backup_project', the walk skips ~/backup_project/ (which contains backups/), eliminating the recursion. The /home backup then completes at its true size and the disk no longer fills. A more general defence is to place the backup destination *outside* of any directory being backed up (for example /mnt/backups or an external volume).

8. Challenges Faced & Resolutions

#	Challenge	Root Cause	Resolution
1	/home backup failed	Recursive copy loop — destination sits inside /home	`Use --exclude='backup_project'`
2	Disk filled to 100%	43 GB runaway archive from the recursion	Deleted the bloated archive to free space
3	Compression failed	No free space left when creating tar.gz	Freed space, then exclude prevents recurrence
4	Integrity check failed	Archive truncated (gzip: unexpected EOF)	Re-run after fix produces a complete archive
5	crontab: /tmp full	crontab -e writes a temp file; disk was full	`Installed cron via echo | crontab - pipe`

Challenge 5 is worth expanding: because the disk was 100% full, `crontab -e` could not create its temporary working file, producing *"No space left on device"*. This is documented in Section 10.

9. Backup Verification (Checksum & Integrity)

The script generates a SHA-256 checksum so any later restore can be validated against the original. The checksum recorded for this run was:

```
115303faf30056364e0232fc0354d6b4c2a6a17fdb5b750d4bfec13a1848b83c  \
        backup_20260614_111409.tar.gz
```

A checksum only proves the file matches its recorded hash; it does ***not*** prove the archive is internally valid. The separate **integrity check** (re-reading the tar) is what caught the corruption here:

```
2026-06-14 11:50:43 [ERROR] INTEGRITY CHECK FAILED: backup_20260614_111409.tar.gz
gzip: stdin: unexpected end of file
tar: Unexpected EOF in archive
tar: Error is not recoverable: exiting now
```

The `unexpected end of file` error confirms the archive was truncated because the disk filled mid-compression. After applying the `--exclude` fix, a re-run produces a complete archive whose integrity check passes.

10. Scheduling with Cron

The backup is scheduled to run automatically every day at **02:00 AM** using cron. The intended crontab entry is:

```
0 2 * * * sudo bash /home/kalibaba/backup_project/backup.sh \
        >> /home/kalibaba/backup_project/cron.log 2>&1
```

The trailing **2>&1** redirects `stderr` (file descriptor 2) into `stdout` (descriptor 1), so both normal output and errors are captured in `cron.log`.

10.1 The /tmp-Full Problem

The first attempt to edit the crontab interactively failed. Because the disk was still full from the runaway backup, `crontab -e` could not write its temporary file:

```
$ crontab -e
crontab: installing new crontab
crontab: crontabs/tmp.xi2QTi: No space left on device
crontab: edits left in /tmp/crontab.hKcCbG/crontab

$ crontab -l
no crontab for kalibaba
```

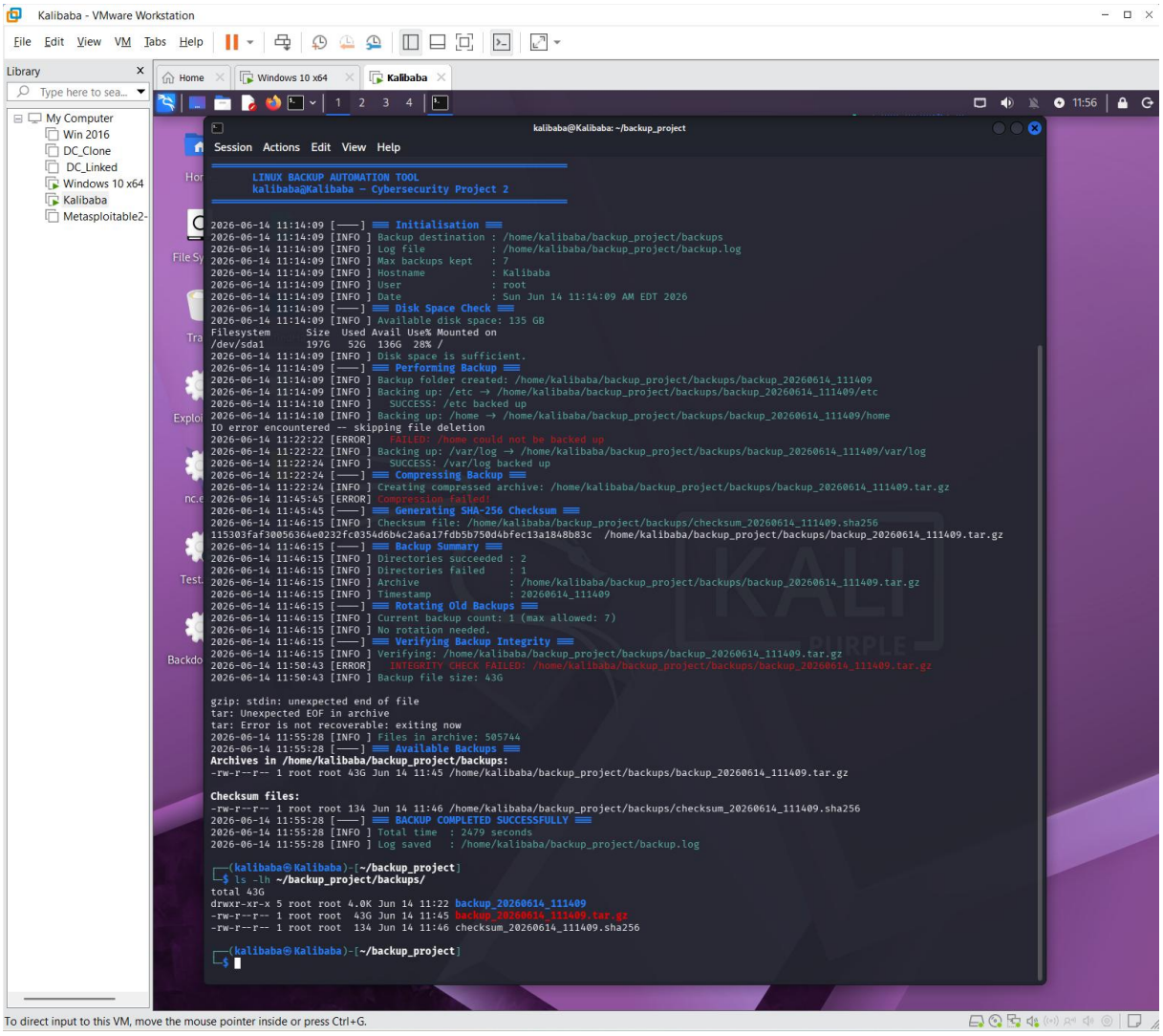

Figure 7. *crontab -e fails with "No space left on device" while the disk is full.*

10.2 The Fix — Pipe Method

Instead of the interactive editor, the cron entry was installed by **piping** it directly into `crontab -`, which avoids the large temp-file workflow. The entry was then confirmed with `crontab -l`:

```
echo "0 2 * * * sudo bash /home/kalibaba/backup_project/backup.sh \
        >> /home/kalibaba/backup_project/cron.log 2>&1" | crontab -

$ crontab -l
0 2 * * * sudo bash /home/kalibaba/backup_project/backup.sh \
        >> /home/kalibaba/backup_project/cron.log 2>&1
```

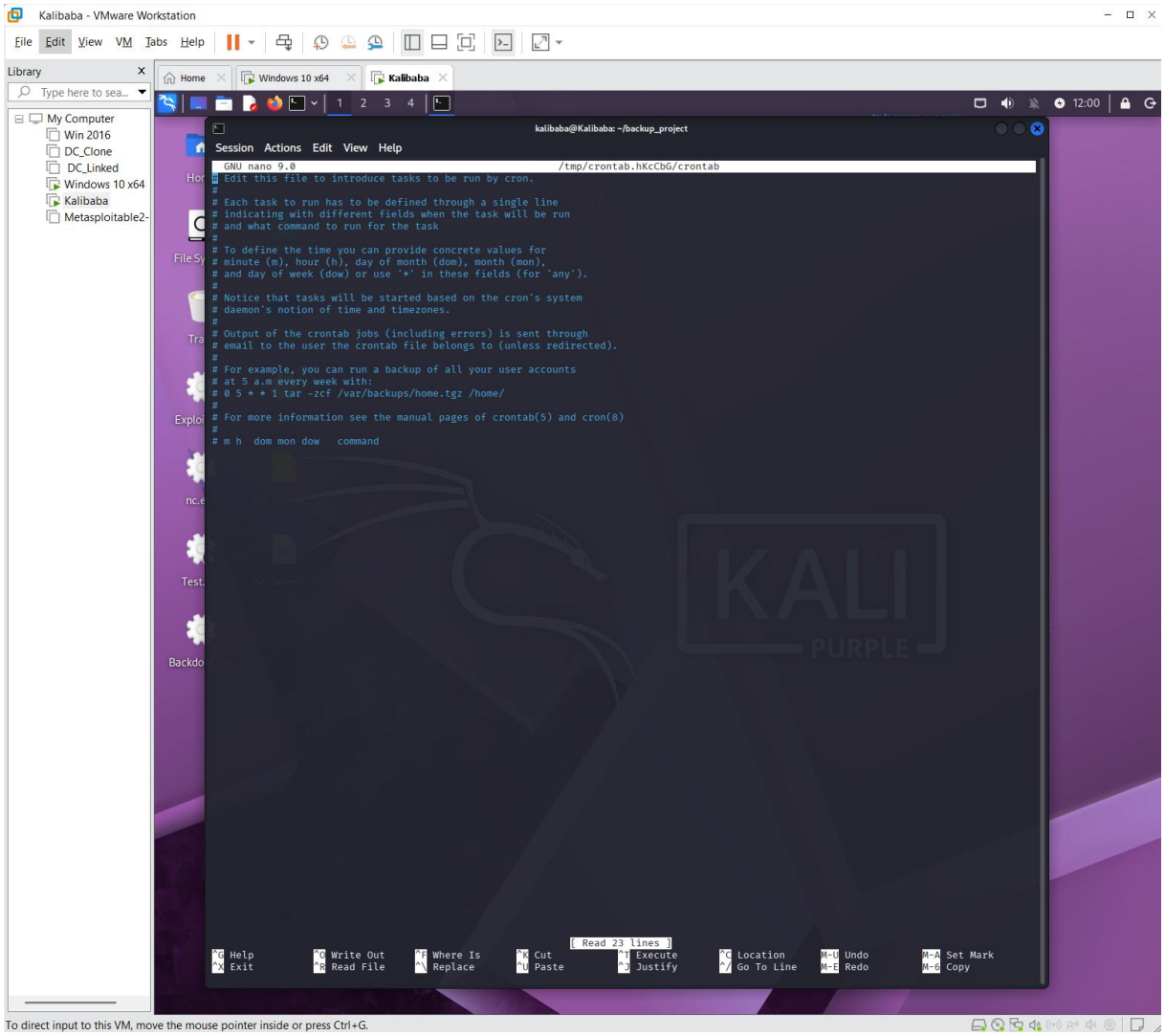

Figure 8. *The pipe method installs the cron job; crontab -l confirms the daily 02:00 schedule.*

11. Restoration Procedure

A backup is only useful if it can be restored. The following procedure restores files from a verified archive.

11.1 Step 1 — Verify the Archive First

Before restoring, confirm the archive matches its recorded checksum and is not corrupt:

```
# Confirm the archive matches its recorded SHA-256
cd ~/backup_project/backups/
sha256sum -c checksum_20260614_111409.sha256

# Confirm the archive is internally valid (lists contents)
tar -tzf backup_20260614_111409.tar.gz > /dev/null && echo "Archive OK"
```

11.2 Step 2 — Restore to a Safe Location

Always extract to a **staging directory** first — never straight over live system files — then copy only what is needed:

```
# Extract into a staging area
mkdir -p ~/restore_staging
tar -xzf backup_20260614_111409.tar.gz -C ~/restore_staging

# Review, then selectively restore (example: /etc/hosts)
sudo cp ~/restore_staging/backup_.../etc/hosts /etc/hosts
```

11.3 Step 3 — Full-Directory Restore (if required)

To restore an entire directory, preserving permissions and ownership, run tar as root:

```
sudo tar -xzpf backup_20260614_111409.tar.gz \
        -C /  --strip-components=1  path/inside/archive
```

The **-p** flag preserves permissions and the **-z** flag handles the gzip layer. Always validate the restore against the checksum before trusting it in production.

12. Future Improvements

Several enhancements would harden this tool for real-world use:

- **Store backups off the source volume —** Place the destination outside any backed-up path (e.g. /mnt/backups or an external/NFS volume) so recursion is structurally impossible.
- **Incremental & differential backups —** Use rsync --link-dest or tar --listed-incremental to back up only changed files, drastically cutting time and space versus full backups every run.
- **Encryption at rest —** Pipe the archive through gpg or openssl so backups of sensitive /etc and /home data are encrypted.
- **Remote/off-site copy —** Push verified archives to remote storage (rsync over SSH, rclone to cloud) so a single-disk failure does not lose all backups (3-2-1 rule).
- **Pre-flight size estimation —** Run du -sh on each target and compare against df free space before starting, aborting early if the backup would not fit.

- **Email / webhook alerting —** Notify the administrator on failure (e.g. via mail or a Slack/Discord webhook) instead of relying on log inspection.
- **Configurable targets —** Move the backup directory list into a config file rather than hard-coding it in the script.
- **Retention by age and count —** Rotate by both count (keep 7) and age (delete anything older than 30 days) for predictable disk usage.

13. Conclusion

This project produced a functional Bash backup tool covering the full lifecycle: disk checks, archiving, compression, checksums, rotation, integrity verification, and cron scheduling. More importantly, it surfaced and **correctly diagnosed** a realistic failure: a recursive copy loop caused by nesting the backup destination inside a backed-up directory. That single design flaw cascaded into a 43 GB archive, a full disk, failed compression, a failed integrity check, and even a broken `crontab -e`. Applying `--exclude='backup_project'` breaks the loop, and moving the destination off the source volume prevents the class of bug entirely. The added restoration procedure and future-improvement roadmap turn a working script into a maintainable, recoverable backup solution.

Appendix A — Full Script Source (backup.sh)

The listing below is the corrected reference implementation, incorporating the `--exclude` fix for the /home recursion issue.

```
#!/bin/bash
# ============================================================
#  LINUX BACKUP AUTOMATION TOOL
#  kalibaba@Kalibaba - Cybersecurity Project 2
# ============================================================

set -o pipefail

# ---- Configuration ----
DEST_ROOT="$HOME/backup_project/backups"
LOG_FILE="$HOME/backup_project/backup.log"
MAX_BACKUPS=7
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DEST="$DEST_ROOT/backup_$TIMESTAMP"
ARCHIVE="$DEST_ROOT/backup_$TIMESTAMP.tar.gz"
TARGETS=("/etc" "/home" "/var/log")

# Exclude the backup project dir to prevent a recursive loop
EXCLUDE="backup_project"

log() { echo "$(date '+%F %T') [$1] $2" | tee -a "$LOG_FILE"; }

# ---- Initialisation ----
mkdir -p "$DEST"
log INFO "Backup destination : $DEST_ROOT"
log INFO "Log file           : $LOG_FILE"
```

```
log INFO "Max backups kept   : $MAX_BACKUPS"

# ---- Disk Space Check ----
AVAIL=$(df -BG --output=avail / | tail -1 | tr -dc '0-9')
log INFO "Available disk space: ${AVAIL} GB"
[ "$AVAIL" -lt 5 ] && { log ERROR "Not enough disk space"; exit 1; }

# ---- Performing Backup ----
ok=0; fail=0
for dir in "${TARGETS[@]}"; do
    log INFO "Backing up: $dir"
    if rsync -a --exclude="$EXCLUDE" "$dir/" "$DEST${dir}/" 2>>"$LOG_FILE"; then
        log INFO "  SUCCESS: $dir backed up"; ((ok++))
    else
        log ERROR "FAILED: $dir could not be backed up"; ((fail++))
    fi
done

# ---- Compressing Backup ----
log INFO "Creating compressed archive: $ARCHIVE"
tar -czf "$ARCHIVE" -C "$DEST_ROOT" "backup_$TIMESTAMP" \
    && log INFO "Compression complete" \
    || log ERROR "Compression failed!"

# ---- SHA-256 Checksum ----
CHECK="$DEST_ROOT/checksum_$TIMESTAMP.sha256"
```

```
sha256sum "$ARCHIVE" > "$CHECK"
log INFO "Checksum file: $CHECK"

# ---- Rotating Old Backups ----
mapfile -t archives < <(ls -1t "$DEST_ROOT"/*.tar.gz 2>/dev/null)
if [ "${#archives[@]}" -gt "$MAX_BACKUPS" ]; then
    for old in "${archives[@]:$MAX_BACKUPS}"; do rm -f "$old"; done
fi

# ---- Verifying Backup Integrity ----
if tar -tzf "$ARCHIVE" >/dev/null 2>&1; then
    log INFO "Integrity check PASSED"
else
    log ERROR "INTEGRITY CHECK FAILED: $ARCHIVE"
fi

log INFO "Directories succeeded : $ok"
log INFO "Directories failed    : $fail"
log INFO "BACKUP COMPLETED"
```